

Nhật ký làm việc — CS2309 SwiftEdit

Ghi chép tiến độ đề tài. Tự động cập nhật bởi skill `.cursor/skills/update-readme-progress/`. Đồng bộ tóm tắt sang [SwiftEdit DeTai CS2309.md](#) Mục 8.1.

Tóm tắt nhanh

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
25026-08-18	Viết báo cáo tổng kết nộp GV (report/REPORT.md)	Mẫu GV: thành viên (placeholder), I/O, phương pháp theo nhật ký, bảng T4/Mac + hình Fig.2/teaser/FP16-FP4/PSNR extreme; Paper demo 1 kết quả; LoRA chưa train	Mac (viết tài liệu)
25026-07-26	Trả lời 4 góp ý thầy: metrics / Paper UI / nén / LoRA pilot	Audit PSNR (lớp A FP16↔FP32); PieBench lớp B summary; Paper demo tab Gradio Mac+T4; luận điểm nén; LORA_DAYNIGHT_PILOT + train/eval scripts (chưa train thật)	Mac (tài liệu + code)

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 5 0 2 6 - 0 7 - 2 4	Viết báo cáo chuyên đề SwiftEdit-RT (report/BA0_CA0_SwiftEdit.md)	Draft đầy đủ §1–10 + 4 flowchart Mermaid; số liệu chính từ FULL_COMPARE_REPORT (fp16 cold 1.39×, PSNR 48.47 dB; EditCache hit −17.3%; FP4 PSNR 21.17 dB); disk FP16 −49.5%; Hybrid multi-turn SSIM 0.959	Mac (viết tài liệu)
2 4f 0 2 6 - 0 7 - 2 2	Hybrid Multi-turn Editing: 3 candidate, Regen/Pick/Undo và local full-resolution composite	Inference trả clean latent + mask; latent jitter 0.05 tạo khác biệt pairwise MAE ~0.012; benchmark 5 lượt: PSNR ngoài mask Naive 30.09 dB vs Hybrid vô hạn, SSIM 0.8353 vs 0.9592, LPIPS 0.0575 vs 0.0119; UI Gradio render OK	Mac M4 (MPS); fp16; gradio 5.50
2 4e 0 2 6 - 0 7 - 1 9	Pause tinh chỉnh RT; note hướng app xóa vật thể	Dừng FP4/xFormers/ToMe/TensorRT cho đến khi đủ bundle + viết đánh giá precision; app object-removal = phase sau (chưa code); ghi FP4_DECISION §0 + PRECISION_CASES	Mac (tài liệu); Colab (user test)

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4e 0 2 6 - 0 7 - 1 9	Thêm fp16_weight_xformers (xFormers MEA trên FP16 disk)	Config mới fp16_disk_xformers; giữ nguyên fp16_weight; MEA qua Diffusers (inverse) + efficient_attention (gen self-attn); notebook SELECT mặc định xformers; chờ bundle T4	Mac (code); Colab T4 (chờ eval)
2 4e 0 2 6 - 0 7 - 1 9	Đóng số FP4 trên T4; xếp hạng hướng RT tiếp theo	Dừng FP4 làm tối ưu chính (ablation âm); không lưu checkpoint FP4; baseline fp16+EditCache; plan xFormers→ToMe→TensorRT tại experimental_data/FP4_DECISION_AND_NEXT_PLAN.md	Mac (tài liệu); Colab T4 (quan sát)
2 4e 0 2 6 - 0 7 - 1 9	Precision picker: workflow 3 lần + fp4_weight (fp4_from_fp16) end-to-end trên notebook	Catalog/eval alias fp4_weight→quant=fp4; notebook lần 3 reuse fp16 + bitsandbytes; PRECISION_CASES workflow fp32→fp16_weight→fp4_weight; chờ bundle T4 full 600 để ghi số liệu so sánh cuối	Mac (code/docs); Colab T4 (user eval)

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4e 0 2 6 - 0 7 - 1 9	Phase A: checkpoint fp16 trên disk + eval disk/memory/PSNR (Mac); luồng Colab Drive sẵn sàng	Disk -49.5% (9.79→4.94 GiB); peak load MPS 12366→6567 MB; PSNR_vs_fp32 mean 51.4 dB; scripts convert/eval + notebook Drive; bundle experimental_data/precision_disk_vram_2026-07-19/	Mac MPS; torch 2.12.0
2 1 0 2 6 - 0 7 - 0 3	Notebook diffusion-from-scratch 01→04; QA F_theta/SBv2/pipeline	4 notebook + 4 script; QA §1-3: 15 câu; pipeline 1+1 (F_theta + SBv2)	Mac; học lý thuyết + notebook .venv
2 5 0 2 6 - 0 6 - 1 8	Rà soát toàn bộ tiến độ đề tài; cập nhật README (mục Tiến trình hiện tại), SwiftEdit_DeTai §8.2	33/80 task (~41%); SwiftEdit-RT mạnh (2400 edit Colab, fp16+cache khuyến nghị); thiếu báo cáo GD5, ablation, slide; ưu tiên: báo cáo → ablation → slide	Mac M4; tài liệu

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4e 0 2 6 - 0 6 - 1 7	TBenchmark precision fp32/fp16/fp8/fp4 (200×3×4 config, Colab T4)	fp16+cache khuyến nghị: 1.70×/1.82×, VRAM −42.1%, PSNR 48.6dB; fp8 1.92× nhưng PSNR 6.0dB (hỏng); fp4 VRAM −48.5%, PSNR 21.7dB; RUN_ID 20260617-0336-bb4785; experimental_data/quality_speed_bench_2026-06-17/	Colab Tesla T4; torch 2.11; git 1a6706c
2 4e 0 2 6 - 0 6 - 1 4	Benchmark tốc độ + VRAM + chất lượng quy mô lớn (fp32 vs fp16+cache)	200 ảnh × 3 prompt = 600 edit/config trên Tesla T4: fp16+cache nhanh 1.70× (overall)/1.82× (cache-hit), giảm 42.1% VRAM (14.6→8.5GB), PSNR 48.5dB / SSIM 0.998 / LPIPS 0.0008 vs fp32 (600 ảnh) → tăng tốc + tiết kiệm bộ nhớ gần như không mất chất lượng; notebook + RUN_ID + zip bằng chứng; experimental_data/quality_speed_bench_2026-06-14/	Colab Tesla T4 (CUDA); torch 2.11
2 4d 0 2 6 - 0 6 - 1 4	Xóa vật thể bằng khoanh vùng (user_mask + tab “Xóa vật thể”)	user_mask ghi đề self-guided mask trong edit_image; UI gr.ImageEditor vẽ cọ; xóa OK headphones (vật nhỏ/vừa, ~6s), kiểm chứng mask khoanh đúng vùng; vật rất lớn (xe đạp ~39% khung) còn sót — SwiftEdit không phải inpainting chuyên dụng; lưu experimental_data/object_removal_2026-06-14/	Mac M4 (MPS); .venv

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4f 0 2 6 - 0 6 - 1 4	Demo UI Gradio (app_gradio.py) tích hợp fp16 + channels_last + EditCache	Self-test OK (ảnh edit đúng, ~7.8s edit đầu gồm compile); hiện runtime + dtype + trạng thái cache; chạy local 127.0.0.1:7860	Mac M4 (MPS); .venv; gradio 5.50
2 4a 0 2 6 - 0 6 - 1 4	fp16 + channels_last (VAE giữ fp32) cho SwiftEdit	fp16 nhanh ~3.3× (máy nguội) → ~7× (chạy liên tục, fp32 throttle); PSNR ~45dB vs fp32, không NaN/đen; tác động end-to-end lớn nhất	Mac M4 (MPS); .venv
2 4a 0 2 6 - 0 6 - 1 4	Cache latent + CLIP image embed + source prompt embed (EditCache + embed_cache)	Tiết kiệm ~9.93s/edit ở stage phụ thuộc ảnh/source (gen_image_embeds -11.5s, vae_encode -0.95s); embed deterministic (allclose); callers cũ không đổi	Mac M4 (MPS); .venv

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4d 0 2 6 - 0 6 - 1 4	Vectorized self-guided mask trên GPU (bỏ .cpu().apply_)	mask_estimate 12.2ms→4.6ms (~2.6×, tiết kiệm ~7.6ms/ảnh); mask giống hệt baseline; chỉ ~0.02% tổng pipeline (~72s/ảnh) nên runtime end-to-end ~không đổi	Mac M4 (MPS); .venv
2 3c 0 2 6 - 0 6 - 1 4	Đo timing từng công đoạn (StageTimer) + eval PIE-Bench subset 20 mẫu	20 mẫu/Apple M4 MPS: TB 69.0s/ảnh (steady 73.6s); UNet x2 ~43%, IP embeds ~24%, VAE decode ~23%; CLIP-Whole 23.02, CLIP-Edited 21.46, PSNR nền 14.01 (9/20); lưu experimental_data/piebench_subset20_2026-06-14/	Mac M4 (MPS); .venv; torch 2.12.0
2 4d 0 2 6 - 0 6 - 0 5	Đề xuất hướng ứng dụng xóa vật thể / inpainting	Cập nhật đề tài/README/Overview/QA: dùng SwiftEdit để xóa object bằng prompt + self/user mask; đánh giá khả thi trung bình-cao với object nhỏ/vừa; metric detector confidence drop, CLIP margin, PSNR/SSIM/LPIPS ngoài mask, realism/human rating; so LaMa nếu kịp.	Mac M4; tài liệu

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4c 0 2 6 - 0 6 - 0 5	Đề xuất hướng ứng dụng global style/weather edit	Cập nhật đề tài/README/Overview/QA: dùng SwiftEdit cho ngày↔đêm, mùa, mưa↔nắng; đánh giá khả thi trung bình; bỏ mask metric, dùng CLIP target, zero-shot CLIP label, DINO/CLIP image similarity, LPIPS/SSIM phụ, IQA/human rating, FID/KID nếu có target domain.	Mac M4; tài liệu
2 4d 0 2 6 - 0 6 - 0 5	Chốt hướng SwiftEdit-RT đầu tiên: bỏ decode noise_image	Patch IPSBV2Model.gen_img() thêm return_noise_image=False, mặc định không decode ảnh nhiễu không dùng; cập nhật README/QA/ đề tài với ranking ưu tiên các hướng tăng tốc không train.	Mac M4; code + tài liệu
2 4d 0 2 6 - 0 6 - 0 5	Đổi hướng đào sâu sang SwiftEdit-RT realtime inference acceleration	Cập nhật đề tài/README/Overview/QA: loại SAM 3 khỏi pipeline chính vì tăng latency; chọn hướng profile bottleneck, GPU mask threshold, bỏ decode thừa, cache latent/embedding, thử fp16/channels_last/torch.compile và TinyVAE/TAESD.	Mac M4; tài liệu

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 4d 0 2 6 - 0 6 - 0 5	Khảo sát ban đầu: SwiftEdit + SAM 3 concept-guided mask replacement	Đã từng cân nhắc SAM 3 để thay self-guided mask; sau phân biện realtime, hướng này chuyển thành optional/offline mask analysis thay vì hướng chính.	Mac M4; tài liệu
2 3c 0 2 6 - 0 6 - 0 5	Chuẩn hóa tài liệu đánh giá SwiftEdit/PieBench và sửa CLIP-Whole trong metric code	Ghi rõ PSNR/MSE vùng nền, CLIP-Whole/Edited theo edit prompt, runtime; nguồn: PIE-Bench/PnP Inversion ICLR 2024, SwiftEdit CVPR 2025, CLIP/CLIPScore. <code>piebench_metrics.py</code> đã sửa <code>clip_whole</code> dùng <code>edit_prompt</code> ; compile all OK.	Mac M4 (MPS); .venv
2 3c 0 2 6 - 0 6 - 0 5	Viết scripts đánh giá PIE-Bench (<code>piebench_utils.py</code> , <code>piebench_metrics.py</code> , <code>run_piebench_eval.py</code>)	Smoke eval 1 mẫu chạy OK trên Mac MPS (~50s/ảnh, CLIP-Whole=18.76, CLIP-Edited=22.64); <code>metrics.csv</code> sinh đúng; PIE-Bench đầy đủ tài qua form PnP Inversion.	Mac M4 (MPS); .venv; torchmetrics 1.9.0

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 0 2 6 - 0 6 - 0 5	2a. Chạy preset 02.jpg dog→dog with mouth opened trên Mac MPS (cùng prompt Colab T4)	edit_image 30.1s; results/notebook/nb_dog_dog_to_dog_with_mouth_opened.png; T4 1.3s cùng preset → Mac ~23× chậm hơn T4;	Mac M4 (MPS); pyenv 3.12.10; .venv
2 0 2 6 - 0 6 - 0 4	2b. Chạy notebook CS2309_SwiftEdit_test trên Colab T4 (extension): preset dog→dog wi	edit_image 1.3s; output results/notebook/nb_dog_dog_to_dog_with_mouth_opened.png; so với Mac MPS ~91s (woman) và paper A	Google Colab — Tesla T4 (Colab extension)
2 0 2 6 - 0 6 - 0 4	2b. Patch notebook + requirements (GPU T4, HF stack mới, upload path)	Sẵn sàng chạy Colab extension; fix EncoderDecoderCache/numpy/upload; chưa log runtime T4 OK	Colab extension + Colab web

N g à i đ o y ạ n	Công việc	Kết quả / Ghi chú	Môi trường
2 2a. 0 Ma 2 c 6 - 0 6 - 0 4	Notebook notebooks/CS2309_SwiftEdit_test.ipyn b (preset, upload ipywidgets 8, inf	Notebook chạy OK trên Mac (.venv); upload widget sửa tuple ipywidgets 8	Mac M4 (MPS); Jupyter .venv
2 2a. 0 Ma 2 c 6 - 0 6 - 0 4	Clone SwiftEdit; pyenv 3.12.10 + .venv + requirements-mac.txt (PyTorch MPS)	Demo woman→Taylor Swift OK; output SwiftEdit/result_woman->Taylor Swift.png; ~91s/ảnh trên MPS	Mac M4 (MPS); pyenv 3.12.10; .venv
2 0. 0 Kh 2 ời 6 tạo - pro 0 ject 6 - 0 1	Tạo README, đề tài chi tiết, skill Cursor hỗ trợ nhật ký	Repo CS2309.CH201 sẵn sàng; skill sync README + NHAT_KY + §8.1	Mac M4

Chi tiết theo phiên làm việc

(Các entry chi tiết xuất hiện bên dưới, mới nhất ở trên cùng.)

2026-08-18 — [5] Viết báo cáo tổng kết nộp GV (REPORT.md)

Môi trường: Mac (viết tài liệu)

Công việc đã làm:

- Viết report/REPORT.md theo mẫu nộp của GV (thành viên, tầm quan trọng, I/O, phương pháp, kết quả, ưu/nhược, tài liệu).
- Phần phương pháp bám chuỗi nhật ký: tái hiện → profile → FP16/cache/disk/xFormers/FP4 → eval 600 job → demo Hybrid.
- Copy hình: Fig. 2, teaser, so sánh FP32/FP16/FP4, cặp PSNR thấp/cao vào report/assets/.

Kết quả:

- File nộp: [report/REPORT.md](#). Xuất PDF/DOCX khi nộp; điền bảng thành viên.

Bước tiếp theo:

- Điền họ tên / MSSV; pandoc hoặc Word xuất PDF; nộp kèm slide + GitHub viewer.
-

2026-07-26 — [5] Trả lời 4 góp ý thầy (metrics, Paper UI, nén, LoRA)

Môi trường: Mac (tài liệu + code)

Công việc đã làm:

- Audit PSNR 48.6 (lớp A FP16↔FP32) + PieBench lớp B summary; đổi wording slide/báo cáo/QA.
- Thêm Gradio tab Paper demo (chỉ prompt → 3 kết quả) Mac + T4; đổi tên tab ROI.
- Viết luận điểm nén (fit T4/Mac, headroom, latency, disk).
- Spec + script LoRA day↔night (train_lora_daynight.py, eval_lora_daynight.py, data/daynight_lora/).

Kết quả:

- report/AUDIT_PSNR_FIDELITY.md, EDIT_QUALITY_SUMMARY.md, LORA_DAYNIGHT_PILOT.md.
- LoRA chưa train thật (chờ ảnh Gemini).

Việc tiếp theo: Thu thập 50–150 cặp Gemini → train trên T4 → demo Paper tab trước thầy.

2026-07-24 — [5] Viết báo cáo chuyên đề SwiftEdit-RT

Môi trường: Mac (viết tài liệu)

Công việc đã làm:

- Viết report/BAO_CA0_SwiftEdit.md theo kế hoạch: mở đầu, bài toán, pipeline ngắn + ý tưởng SwiftEdit, phương pháp nghiên cứu, thiết kế thực nghiệm, kết quả tối ưu inference, ứng dụng bổ sung, hạn chế và kết luận.
- Thêm 4 flowchart Mermaid: multi-step vs SwiftEdit; quy trình nghiên cứu; đánh giá công bằng; masked ROI.
- Dùng số liệu bundle cuối (FULL_COMPARE_REPORT.md), disk/memory Phase A, object removal và multi-turn Hybrid.

Kết quả:

- Báo cáo draft đầy đủ ~430 dòng, không dùng LaTeX.
- Kết luận chính: trên T4, FP16 + EditCache là cấu hình cân bằng tốt nhất; FP4 không phù hợp.

Task README đã [x]: Lý do chọn đề tài; Overview + pipeline; I/O; RQ; kế hoạch thực nghiệm; kết quả; so sánh dataset/model; kết luận; tài liệu tham khảo; Overview + Related Work trong báo cáo.

Bước tiếp theo:

- Ablation s_edit/s_non-edit; slide trình bày; baseline TurboEdit nếu còn thời gian.

2026-07-22 — [4f] Hybrid Multi-turn Editing giữ chất lượng qua nhiều lượt

Môi trường: Mac M4 (MPS); fp16; Gradio 5.50

Công việc đã làm:

- Thêm phiên chỉnh sửa có master full-resolution, history, Undo và batch candidate bất biến.
- Thêm workflow sinh tuần tự 3 candidate trong một background task dùng chung cache; UI hiển thị ngay từng ảnh, hỗ trợ Regen, chọn ảnh để commit rồi tiếp tục chỉnh.
- Thay resize/letterbox toàn ảnh bằng masked ROI kiểu A1111: user tô mask, backend tự tạo crop vuông có context để infer 512×512, rồi chỉ blend vùng mask về master; pixel ngoài mask không đổi.

- Mở rộng inference để trả clean latent, source latent và mask; thêm latent jitter có seed.
- Ảnh xạ mask latent về ảnh gốc và chỉ composite vùng local edit; global edit được đánh dấu giới hạn 512×512.
- Thêm unit test và script benchmark Naive so với Hybrid qua 5 vòng VAE.

Kết quả:

- VAE seed riêng tạo khác biệt quá nhỏ (pairwise MAE 0.0007–0.0013); latent jitter 0.05 đạt khoảng 0.012.
- Sau 5 lượt, Naive đạt PSNR ngoài mask 30.09 dB, SSIM 0.8353, LPIPS 0.0575.
- Hybrid giữ nguyên pixel ngoài mask (PSNR vô hạn), SSIM 0.9592 và LPIPS 0.0119.
- Unit test 5/5 pass; UI có đủ Tạo 3 kết quả, Regen, Pick và Undo.

Bước tiếp theo:

- Test thủ công thêm nhiều prompt và kích thước ảnh; chỉ dùng global mode khi chấp nhận giới hạn 512×512.

2026-07-19 — [4e] Pause tinh chỉnh RT; hướng app tiếp theo

Môi trường: Mac (tài liệu); Colab T4 (user chạy hết test)

Công việc đã làm:

- Chốt **PAUSED** đóng góp tinh chỉnh (FP4 / xFormers / ToMe / TensorRT): đủ hướng đề đo; không implement thêm
- Note: sau đủ bundle → viết **đánh giá** phần precision/RT; phase sau = app **xóa vật thể nhỏ** (chưa scaffold)
- Cập nhật [experimental_data/FP4_DECISION_AND_NEXT_PLAN.md](#) §0 + [PRECISION_CASES.md](#)

Kết quả:

- Tài liệu ghi rõ: đang chờ số liệu / thinking; không mở ToMe/TRT lúc này

Bước tiếp theo:

- User hoàn tất Colab + tải bundle
 - Local: `compare_precision_runs.py` → viết đánh giá phần này
 - Sau đó mới bàn app object removal
-

2026-07-19 — [4e] Thêm fp16_weight_xformers (xFormers MEA)

Môi trường: Mac (code); Colab T4 (user chạy eval)

Công việc đã làm:

- Catalog: fp16_disk_xformers + alias fp16_weight_xformers (**không** dùng fp16_weight / fp16_disk)
- src/efficient_attention.py + wire AttnProcessor/IP (self-attn); Inverse dùng Diffusers enable_xformers_*
- Không gọi Diffusers enable trên gen UNet (tránh ghi đè IP processors / ARaM)
- Notebook SELECT mặc định fp16_weight_xformers; pip install xformers khi chọn

Kết quả:

- Code sẵn sàng đo so với fp16_weight cùng cây disk fp16
- Cross-attn có mask controller vẫn einsum (ARaM) — MEA chủ yếu inverse + self-attn gen

Bước tiếp theo:

- Colab: SELECT fp16_weight_xformers, MAX_JOBS=600 → bundle
- So với bundle fp16_weight: s/edit, VRAM, PSNR

2026-07-19 — [4e] Đóng sổ FP4; xếp hạng hướng RT tiếp theo

Môi trường: Mac (tài liệu); Colab T4 (quan sát GPU RAM ~10.5GB lúc fp4_weight)

Công việc đã làm:

- Tiếp nhận phản biện: bitsandbytes FP4 trên Turing không native; dequant → tốc độ $\approx$ fp16
- Ghi quyết định + plan: [experimental_data/FP4_DECISION_AND_NEXT_PLAN.md](#)
- Cập nhật PRECISION_CASES (case 6 = ablation âm), PRECISION_QUANT context, HUONG_PHAT_TRIEN (thêm xFormers/ToMe; đánh dấu dùng FP4)
- QA: vì sao FP4 fail trên T4; thứ tự thử sau FP4

Kết quả:

- **Dừng** tối ưu FP4 trên T4; không lưu checkpoint FP4 trên disk

- Baseline vận hành: **FP16 + EditCache** (fp16 / fp16_weight)
- Xếp hạng tiếp: xFormers MEA → ToMe → (tùy chọn) TensorRT FP16

Bước tiếp theo:

- Audit attention backend trên Colab (SDP / xFormers)
 - Thử xFormers rồi ToMe; đo s/edit, VRAM, PSNR so fp16+cache
-

2026-07-19 — [4e] Precision picker: fp4_weight (lần 3)

Môi trường: Mac (code/docs); Colab T4 (user chạy eval)

Công việc đã làm:

- Xác nhận fp4_weight → fp4_from_fp16 (quant=fp4, cần fp16 disk)
- Notebook: workflow 3 lần; SELECT mặc định fp4_weight; check bitsandbytes khi chọn fp4
- Cập nhật [experimental data/PRECISION_CASES.md](#)

Kết quả:

- Pipeline code sẵn sàng cho lần 3 trên T4 (reuse fp16 sau lần 2, không convert lại nếu đã có)
- Chưa có số liệu full 600 edit — chờ bundle.zip Colab

Bước tiếp theo:

- Colab: SELECT fp4_weight, MAX_JOBS=600 → bundle
- Local: compare_precision_runs.py với bundle fp32 + fp16_weight + fp4_weight

2026-07-19 — [4e] Phase A: fp16 disk + eval Mac MPS

Môi trường: Mac MPS; torch 2.12.0

Công việc đã làm:

- scripts/convert_weights_fp16.py — xuất UNet inverse + SBv2 + IP-Adapter → SwiftEdit/swiftdit_weights_fp16/

- Patch models.py: from_pretrained(..., torch_dtype=...) + cast IP state_dict
- scripts/run_precision_disk_vram_eval.py — đo disk/memory/PSNR, xuất CSV + COMPARE_WITH_PREV.md + bundle.zip
- scripts/link_weights_fp16_drive.py + notebook CS2309_SwiftEdit_precision_disk_vram.ipynb (Phase B Colab)

Kết quả:

- Disk UNet+IP: **9.79 → 4.94 GiB (-49.5%)**
- Peak load (MPS): fp32 **12366 MB** → fp16_disk **6567 MB**
- PSNR vs fp32: mean **51.4 dB** (min 48.5) trên 2×2 edits
- Thời gian edit mean: fp32 ~31s → fp16_disk ~5.3s (MPS)

Bước tiếp theo:

- Nén/upload swiftedit_weights_fp16 lên Drive MyDrive/CS2309/
- Phase B Colab T4: --configs baseline_fp32, fp16_disk, fp4_from_fp16 + tải bundle.zip

2026-07-03 — [1] Học diffusion-from-scratch + QA F_theta/SBv2/pipeline

Môi trường: Mac; học lý thuyết + notebook .venv

Công việc đã làm:

- Hoàn thành notebook [learn/diffusion-from-scratch/](#) 01→04 (forward, train DDPM, sample DDPM/DDIM); sửa matplotlib Agg; công thức forward trong README
- Thảo luận & ghi QA: F_theta Stage 1/2, SBv2↔eps, notation pipeline edit, SwiftEdit vs DDIM, core idea thay random noise

Kết quả:

- 4 notebook + 4 script; QA mục 1: 4 câu, mục 2: 6 câu, mục 3: 5 câu
- Nắm pipeline 1+1: F_theta invert + SBv2(input_sb, dst_prompt, IP-Adapter, ARaM); Stage 2 DISTS = tái tạo ảnh gốc (không train edit)

Task README đã đánh [x]:

- Tìm hiểu SwiftBrushv2, DDIM inversion, IP-Adapter
- Vẽ/tóm tắt pipeline inversion → mask → ARaM

Bước tiếp theo:

- Đọc paper SwiftEdit Sec. 3–5 (PDF)
 - Vẽ sơ đồ pipeline cho slide/báo cáo (ASCII/Mermaid)
 - Bảng Related Work Task 5
 - Bắt đầu draft báo cáo GD5
-

2026-06-18 — [5] Tổng hợp tiến trình & ưu tiên việc tiếp theo

Môi trường: Mac M4; tài liệu

Công việc đã làm:

- Rà soát toàn bộ tiến độ đề tài; cập nhật README (mục Tiến trình hiện tại), SwiftEdit_DeTai §8.2 với số liệu thật

Kết quả:

- 33/80 task (~41%); SwiftEdit-RT mạnh (2400 edit Colab, fp16+cache khuyến nghị); thiếu báo cáo GD5, ablation, slide; ưu tiên: báo cáo → ablation → slide

Task README đã đánh [x]:

- *(không đánh task README)*

Bước tiếp theo:

- Viết báo cáo GD5
 - Ablation hyperparameter Mac
 - Slide trình bày
 - PIE-Bench 50 mẫu Colab (nếu còn thời gian)
-

2026-06-17 — [4e] Benchmark precision fp32 / fp16 / fp8 / fp4 (Colab T4)

Môi trường: Google Colab — Tesla T4 (CUDA); torch 2.11.0+cu128; diffusers 0.35.2; git 1a6706c

Công việc đã làm:

- Chạy notebook CS2309_SwiftEdit_quality_speed_bench.ipynb với 4 config: baseline_fp32, improved_fp16_cache, improved_fp8_cache (torchao weight-only), improved_fp4_cache (bitsandbytes NF4).
- 200 ảnh × 3 prompt × 4 config = 2400 lần edit đo thời gian; 1800 cặp chất lượng (3 improved vs fp32).

Kết quả chính:

Config	Speedup	VRAM max	PSNR vs fp32	Kết luận
fp16 + cache	1.70× (hit 1.82×)	8446 MB (−42.1%)	48.6 dB	Khuyến nghị
fp8 + cache	1.92× (hit 2.06×)	7819 MB (−46.4%)	6.0 dB	Nhanh nhưng hỏng chất lượng
fp4 + cache	1.68×	7515 MB (−48.5%)	21.7 dB	VRAM thấp nhất; chất lượng giảm nhiều

- Tách fp16/cache (giống lần 06-14): fp16 không cache 1.93s (**1.51×**); cache-hit 1.60s (**1.21×** trên fp16).
- fp32 baseline: 2.91s/edit (lần này hơi chậm hơn lần 06-14 do 2.54s — biến thiên Colab bình thường).

Kết luận nghiên cứu: fp16 + cache là sweet spot. fp8 weight-only không phù hợp SwiftEdit (PSNR 6 dB). fp4 chỉ khi bắt buộc tiết kiệm VRAM.

Dữ liệu:

- Bản nhẹ (commit): experimental_data/quality_speed_bench_2026-06-17/
- Bản đầy đủ (~503MB): results/quality_speed_bench_2026-06-17_20260617-0336-bb4785/

2026-06-14 — [4e] Benchmark tốc độ + VRAM + chất lượng (fp32 vs fp16+cache, Colab T4)

Môi trường: Google Colab — Tesla T4 (CUDA); torch 2.11.0+cu128; diffusers 0.35.2; transformers 4.57.6

Công việc đã làm:

- Viết notebook CS2309_SwiftEdit_quality_speed_bench.ipynb (chạy được Mac + Colab): so bản gốc fp32 (ground truth) vs cải thiện fp16 + cache, N ảnh cấu hình được, mỗi ảnh 3 prompt, chạy tuần tự.
- Đo: tốc độ (tách cache-miss/hit), **VRAM per-edit** (max/min/mean), chất lượng PSNR/SSIM/LPIPS/MSE (fp32 = ground truth).

- Tự dựng dataset PIE-Bench từ parquet HuggingFace (~68MB) theo N (cap 700); RUN_ID/UUID mỗi phiên; đóng gói zip bằng chứng (report + run_meta.json + CSV + ảnh in/out + notebook đã chạy).

Kết quả (200 ảnh × 3 prompt = 600 edit/config):

- Tốc độ: fp16+cache **2.54s** → **1.50s** (overall **1.70×**); cache-hit **1.40s (1.82×**), cache-miss 1.69s.
- Tách đóng góp (2 tầng, dùng prompt đầu = cache-miss của 200 mẫu): **tầng 1 fp16** → **1.50×** vs **fp32** (2.54→1.69s); **tầng 2 cache** → **1.21×** vs **fp16 không cache** (1.69→1.40s, phụ thuộc fp16, −17.3%, 0.29s/edit) ⇒ gộp **1.82×** vs fp32.
- Kiểm chứng baseline: clone SwiftEdit gốc (Qualcomm-AI-research) và diff → baseline fp32 tương đương số học bản gốc (nhánh tối ưu đều tắt/no-op ở fp32); thay đổi thực chất duy nhất là dùng mirror SD2.1-base Manojb/ . . . (trùng trọng số, do repo stabilityai bị gỡ). Bằng chứng: fp32_baseline_verification.md + upstream_diff.md.
- VRAM: **14.6GB** → **8.5GB** (giảm **42.1%**) — đây là lý do fp32 từng sát ngưỡng OOM trên T4 (15GB) còn fp16 dư thoải mái.
- Chất lượng vs fp32: PSNR **48.5dB** (min 34.2), SSIM **0.998** (min 0.987), LPIPS **0.0008** (max 0.0099), MSE 2e-5 trên cả **600** ảnh → khác biệt không thấy được bằng mắt.

Kết luận: cải thiện cho **3 lợi ích đồng thời** — nhanh hơn ~1.7×, giảm ~42% VRAM, chất lượng gần như không đổi. (Cache lossless nên chênh lệch chất lượng nếu có là do fp16.)

Task README đã đánh [x]:

- 4e: benchmark quy mô lớn Colab; lập bảng latency breakdown + speedup + peak memory + PSNR/SSIM/LPIPS/MSE.

Dữ liệu:

- Bản nhẹ (commit): experimental_data/quality_speed_bench_2026-06-14/ (report + run_meta + CSV + grid + 4 ảnh mẫu).
- Bản đầy đủ (~503MB, không commit): results/quality_speed_bench_2026-06-14_20260614-1644-fa0711/ (1200 ảnh output + 200 nguồn + notebook đã chạy).

Bước tiếp theo:

- Thử torch.compile / TinyVAE trên CUDA để tách thêm bottleneck VAE/UNet.
-

2026-06-14 — [4d] Xóa vật thể bằng khoanh vùng (user mask)

Môi trường: Mac M4 (MPS); .venv

Công việc đã làm:

- Thêm `prepare_user_mask()` + tham số `user_mask` cho `edit_image` (SwiftEdit/infer.py): mask người dùng vẽ được resize về latent 64×64 và **ghi đè self-guided mask** trong stage `mask_estimate`.
- Thêm tab “Xóa vật thể (khoanh vùng)” vào `scripts/app_gradio.py`: `gr.ImageEditor` cọ vẽ, `extract_editor_mask()` lấy alpha layer thành mask, `run_removal()` chạy edit với `scale_edit $\approx$ 0 + scale_non_edit $\approx$ 1.2`; thêm chế độ `--selftest-removal`.

Kết quả:

- Xóa **headphones** khỏi mèo (vật nhỏ/vừa, mask $\sim 18\%$): earcup 2 bên + vành trên biến mất, $\sim 6s$ /lần.
- Kiểm chứng mask khoanh đúng vùng: tô nửa trái + “a brick wall” $\rightarrow$ chỉ nửa trái đổi, nửa phải giữ nguyên.
- Ca giới hạn: xóa **xe đạp** (vật rất lớn $\sim 39\%$ khung) còn sót — SwiftEdit là editor ngữ nghĩa one-step, không phải inpainting chuyên dụng; thiếu ngữ cảnh nền để tái sinh.
- Lưu `experimental_data/object_removal_2026-06-14/` (report + ảnh source/mask/result hai ca).

Task README đã đánh [x]:

- 4d: cho khoanh vùng (user_mask) + tab UI; chạy SwiftEdit-UserMask; ghi failure case vật lớn.

Vấn đề / cách xử lý:

- `scale_edit=0` để bỏ giữ vật thể gốc ở vùng tô; mask resize bilinear rồi nhị phân theo `mask_threshold`.
- `edit_image` cần đường dẫn file $\rightarrow$ lưu ảnh nền từ editor ra temp PNG trước khi gọi.

Bước tiếp theo:

- Thêm mask dilation chống ghosting; nếu kịp so LaMa baseline cho vật lớn.
 - **Kế hoạch kiểm tra (chờ user):** [experimental_data/object_removal_2026-06-14/KE_HOACH_KIEM_TRA.md](#) — hướng A (test ảnh phù hợp) / hướng B (LaMa).
-

2026-06-14 — [4f] Demo UI Gradio cho SwiftEdit-RT

Môi trường: Mac M4 (MPS); .venv; gradio 5.50

Công việc đã làm:

- Viết `scripts/app_gradio.py`: upload ảnh + source/edit prompt, slider `scale_edit/scale_non_edit/mask_threshold`, checkbox `cache`.
- Tích hợp `fp16 + channels_last + EditCache`; hiển thị ảnh kết quả + `runtime + dtype + trạng thái cache`.
- Thêm `gradio>=5, <6 + pandas` vào `requirements-mac.txt`; thêm chế độ `--selftest` để kiểm thử không cần mở server.

Kết quả:

- Self-test OK: ảnh edit đúng (rusty bicycle), ~7.8s cho edit đầu (gồm compile MPS); các edit sau nhanh hơn nhờ cache.
- Demo chạy local `http://127.0.0.1:7860`; có ví dụ prompt sẵn.

Task README đã đánh [x]:

- 4f: Gradio upload ảnh + prompt; hiện output + runtime; chạy local Mac.

Vấn đề / cách xử lý:

- gradio 6 kéo `huggingface-hub` lên 1.x phá `transformers 4.57` → ghim `gradio>=5, <6 + hf-hub<1.0`.
- `edit_image` trả batch 2 ảnh (source recon + edit) → demo lấy ảnh edit (index cuối); cũng giải thích vì sao ảnh benchmark trước hiện cạnh nhau (do `save_image` ghép grid).
- Sandbox không bind được localhost để test UI → dùng `--selftest` chạy đúng code path.

Bước tiếp theo:

- Lưu output demo vào folder riêng cho slide; thêm nút so sánh `fp16 vs fp32` ngay trong UI.

2026-06-14 — [4a] SwiftEdit-RT: fp16 + channels_last (VAE giữ fp32)

Môi trường: Mac M4 (MPS); .venv

Công việc đã làm:

- Thêm tham số `dtype/channels_last` cho 3 model class (InverseModel, AuxiliaryModel, IPSBV2Model) + helper `resolve_dtype/module_dtype`.
- Cast ranh giới dtype trong `gen_img / edit_image / mask_ip_controller` để chạy fp16; VAE luôn fp32.
- Viết `scripts/bench_dtype.py` đo per-stage + chất lượng (PSNR/MSE vs fp32, phát hiện NaN/ảnh đen).

Kết quả:

- fp16 nhanh $\sim 3.3\times$ (máy nguội, 1 edit) $\rightarrow \sim 7\times$ (chạy liên tục: fp32 bị throttle 35 $\rightarrow$ 53s, fp16 ổn định ~ 5.5 s).
- PSNR ~ 45 dB so với fp32, MSE ~ 0 , **không NaN/ảnh đen** $\rightarrow$ chất lượng gần như không đổi.
- `channels_last` thêm $\sim 5\%$ và làm thời gian phẳng hơn.
- Bằng chứng throttling: stage VAE (fp32 ở mọi config) chạy chậm $\sim 3.9\times$ ở fp32 dù cùng dtype.
- Đây là tối ưu **tác động end-to-end lớn nhất** (tăng tốc toàn bộ UNet inverse+gen + text/image encoder).
- Lưu `experimental_data/fp16_benchmark_2026-06-14/` (report + log + ảnh mẫu fp32/fp16/fp16_cl).

Task README đã đánh [x]:

- 4e: Thử fp16 + `channels_last`.

Vấn đề / cách xử lý:

- fp16 ban đầu vỡ ở mask controller (mask fp32 trộn query/value fp16) $\rightarrow$ cast mask theo dtype runtime.
- VAE giữ fp32 để tránh NaN/ảnh đen (SD VAE fp16 không ổn định).
- Phát hiện bug sẵn: `vae.encode` ép input theo `weight_dtype` $\rightarrow$ sửa dùng `vae.dtype`.
- Wall-clock nhiều thermal throttling $\rightarrow$ dùng median nhiều edit + per-stage; báo cáo cả mức bảo thủ (nguội) và sustained.

Bước tiếp theo:

- `torch.compile` trên CUDA; gắn fp16 + cache vào Gradio demo; đo lại 20 mẫu PIE-Bench với fp16 để xác nhận chất lượng diện rộng.

2026-06-14 — [4a] SwiftEdit-RT: cache latent + CLIP image embed + source prompt embed

Môi trường: Mac M4 (MPS); `.venv`

Công việc đã làm:

- Thêm EditCache (infer.py) + tham số embed_cache (gen_img); split text-encode để cache riêng source prompt; benchmark interleaved khi nhiều nhiệt (scripts/bench_cache.py)

Kết quả:

- Cache tiết kiệm ~9.93s/edit ở stage phụ thuộc ảnh/source (gen_image_embeds -11.5s, vae_encode -0.95s); embed deterministic (allclose=True); wall-clock 56.1->43.7s (nhiều thermal); callers cũ không đổi hành vi

Task README đã đánh [x]:

- (không đánh task README)

Vấn đề / cách xử lý: Wall-clock end-to-end bị thermal throttling che lấp -> chuyển sang đo per-stage; gen_text_encode hiện số âm do nhiều MPS op nhỏ

Bước tiếp theo:

- Cache trong Gradio/realtime demo; gộp 2 lần UNet (inverse+gen) hoặc batch; thử fp16 cho CLIP image encoder
-

2026-06-14 — [4d] SwiftEdit-RT: vectorized self-guided mask trên GPU

Môi trường: Mac M4 (MPS); .venv

Công việc đã làm:

- Thay mask12.detach().cpu().apply_(to_binary) bằng (mask12 > threshold) vectorized trên MPS/CUDA trong infer.py

Kết quả:

- Kiểm thử 5 ảnh PIE-Bench subset (Mac M4/MPS, steady-state):
 - Stage mask_estimate: **12.2 ms** → **4.6 ms** (speedup ~2.6×, tiết kiệm tuyệt đối ~7.6 ms/ảnh ≈ 0.008s).
 - Mask output **giống hệt** baseline (kiểm chứng bằng torch.equal ở nhiều threshold → 0 pixel khác).
 - Stage này chỉ chiếm ~0.02% tổng pipeline (mask 12.2 ms / total ~71.8 s/ảnh) → tối ưu mask **không** giảm runtime end-to-end rõ rệt (≈0.01%).
 - Lợi ích thật: bỏ vòng lặp Python từng pixel + đồng bộ GPU→CPU→GPU; lợi ích tăng theo độ phân giải.
- So với bottleneck thật cùng run 20 mẫu: gen_image_embeds ~23.5%, gen_vae_decode ~23.2%, gen_unet ~21.7%, unet_inverse ~21.5%.

Task README đã đánh [x]:

- *(không đánh task README)*

Bước tiếp theo:

- Cache latent/image embedding (SwiftEdit-RT priority 3)
-

2026-06-14 — [3c] Đo timing từng công đoạn + eval PIE-Bench subset 20 mẫu (Mac M4)

Môi trường: Mac M4 (MPS); .venv; torch 2.12.0

Công việc đã làm:

- Thêm StageTimer (SwiftEdit/timing.py) đo timing từng bước edit_image/gen_img, ghi JSONL; tạo subset 20 mẫu từ HF PIE_Bench_pp; viết summarize_timing.py; fix shape mask trong piebench_metrics.py

Kết quả:

- 20 mẫu/Apple M4 MPS: trung bình 69.0s/ảnh (steady 73.6s); UNet x2 ~43%, IP embeds ~24%, VAE decode ~23%; CLIP-Whole 23.02, CLIP-Edited 21.46, PSNR nền 14.01 (9/20). Lưu experimental_data/piebench_subset20_2026-06-14/

Task README đã đánh [x]:

- Tìm hiểu PieBench benchmark
- Lưu results/piebench/metrics.csv

Vấn đề / cách xử lý: return_noise_image bỏ đi không cải thiện tốc độ (chi ~0.0001s); bottleneck thật là 2x UNet + IP embeds + VAE decode

Bước tiếp theo:

- Chạy PIE-Bench đầy đủ/nhiều mẫu hơn; so sánh baseline
-

2026-06-05 — [4d] Hướng ứng dụng: object removal / inpainting

Môi trường: Mac M4; tài liệu đề tài

Công việc đã làm:

- Thêm hướng dùng SwiftEdit để xóa vật thể khỏi ảnh.
- Tách rõ đây là local edit/inpainting: mask vẫn có ý nghĩa, khác với global style/weather edit.
- Đề xuất cấu hình SwiftEdit-SG, SwiftEdit-UserMask/GTMask, SwiftEdit-SG+Dilate và LaMa baseline nếu còn thời gian.
- Cập nhật SwiftEdit_DeTai_CS2309.md, README.md, SwiftEdit_Overview.md, QA.md.

Kết quả:

- Độ khả thi đánh giá là **trung bình-cao** với object nhỏ/vừa và nền đơn giản.
- Rủi ro chính: object lớn che nhiều background, viền/ghosting, texture nền bị méo; LaMa có thể mạnh hơn vì là inpainting model chuyên dụng.
- Metric đề xuất: detector confidence drop, CLIP margin "without object" vs "with object", PSNR/SSIM/LPIPS ngoài mask, realism/human rating, IoU/Dice mask nếu có GT.

Bước tiếp theo:

- Chọn 20–40 ảnh có object cần xóa.
- Chạy SwiftEdit-SG trước, visualize mask; sau đó thử user/GT mask và mask dilation nếu cần.

2026-06-05 — [4c] Hướng ứng dụng: global style/weather edit

Môi trường: Mac M4; tài liệu đề tài

Công việc đã làm:

- Đánh giá hướng dùng SwiftEdit cho global scene/style editing: ngày↔đêm, mùa xuân/hạ/thu/đông, mưa↔nắng, overcast/golden hour.
- Ghi rõ mask IoU/Dice và PSNR/MSE background không phù hợp vì global edit tác động lên toàn ảnh.
- Đề xuất metric mới theo 3 trục: đúng target/style, giữ content/layout, realism/artifact.
- Cập nhật SwiftEdit_DeTai_CS2309.md, README.md, SwiftEdit_Overview.md, QA.md.

Kết quả:

- Độ khả thi đánh giá là **trung bình**: hợp để khảo sát ứng dụng và failure cases, đặc biệt với edit nhẹ như day→night, sunny→overcast, warm/cold tone.
- Rủi ro cao hơn ở mùa đông/tuyết, mưa lớn, đêm có nguồn sáng/phản chiếu vì cần thay đổi ánh sáng, texture và chi tiết toàn cảnh.

Bước tiếp theo:

- Chọn 20–40 ảnh outdoor/street/landscape.
 - Chạy prompt nhẹ/mạnh; nếu có full-image mask thì so SwiftEdit-SG vs SwiftEdit-FullMask.
 - Tạo bảng metric: CLIP target, zero-shot CLIP label, DINO/CLIP image similarity, LPIPS/SSIM phụ, human rating và runtime.
-

2026-06-05 — [4d] Chốt hướng đầu: bỏ decode noise_image

Môi trường: Mac M4; code + tài liệu

Công việc đã làm:

- Chọn hướng SwiftEdit-RT đầu tiên vì đơn giản nhất và không làm giảm chất lượng: không tạo noise_image trong IPSBV2Model.gen_img() khi caller không dùng.
- Patch SwiftEdit/models.py: thêm cờ return_noise_image=False; chỉ decode noise_image khi bật cờ này.
- Patch SwiftEdit/infer.py: gọi gen_img(..., return_noise_image=False) rõ ràng.
- Cập nhật README.md, QA.md, SwiftEdit_DeTai_CS2309.md với thứ tự ưu tiên các hướng tăng tốc không train.

Kết quả:

- Output ảnh edited chính giữ nguyên công thức decode; chỉ bỏ một VAE decode phụ vốn bị discard bởi res_gen_img, _.
- Thứ tự ưu tiên còn lại: GPU mask threshold → profiler → cache latent/embedding → channels_last/fp16 → torch.compile → TinyVAE/TAESD → TensorRT/Core ML/quantization.

Bước tiếp theo:

- Chạy đo baseline vs SwiftEdit-no-noise-decode trên Mac MPS và Colab T4.
 - Sau đó patch GPU mask threshold để bỏ .cpu().apply().
-

2026-06-05 — [4d] Đối hướng đào sâu: SwiftEdit-RT

Môi trường: Mac M4; tài liệu đề tài

Công việc đã làm:

- Rà lại luận điểm cốt lõi của SwiftEdit: realtime/instant editing.
- Đánh giá lại hướng SAM 3: tuy dễ làm và có định lượng mask, nhưng thêm segmentation model sẽ tăng latency và tài nguyên end-to-end.
- Chọn hướng mới: **SwiftEdit-RT: Realtime-Oriented Inference Acceleration**.
- Cập nhật SwiftEdit_DeTai_CS2309.md, README.md, SwiftEdit_Overview.md, QA.md theo hướng profiling + tối ưu inference.

Kết quả:

- Hướng chính mới gồm: latency breakdown theo module, vectorized self-guided mask trên GPU, bỏ decode noise_image không dùng, cache latent/embedding cho cùng ảnh nhiều prompt, thử fp16, channels_last, torch.compile, TinyVAE/TAESD.
- SAM 3 được hạ xuống optional/offline mask quality analysis, không tính là pipeline realtime.

Bước tiếp theo:

- Viết profiler cho edit_image() để đo từng module.
- Patch nhanh 2 tối ưu ít rủi ro: GPU mask threshold và return_noise_image=False.
- Chạy so sánh baseline vs optimized trên Mac MPS và Colab T4.

2026-06-05 — [4d] Khảo sát ban đầu: SwiftEdit + SAM 3

Môi trường: Mac M4; tài liệu đề tài

Công việc đã làm:

- Khảo sát các hướng thay module pipeline: SAM 3 mask, GroundingDINO+SAM2, auto source prompt bằng Florence-2/Qwen2.5-VL, grounded evaluation, baseline FLUX.1 Kontext/Qwen-Image-Edit/Step1X-Edit.
- Ban đầu chọn hướng **SwiftEdit + SAM 3: Concept-guided Mask Replacement** để phân tích mask.
- Cập nhật SwiftEdit_DeTai_CS2309.md, README.md, SwiftEdit_Overview.md, QA.md với pipeline mới, research questions RQ8/RQ9, đóng góp C13, kế hoạch thí nghiệm và tài liệu tham khảo.

Kết quả:

- Hướng SAM 3 được định nghĩa rõ: trích concept → SAM 3 sinh mask → SwiftEdit ARaM dùng external mask → so sánh self-guided / GT / SAM 3.
- Sau phân biện realtime, hướng này được chuyển thành optional/offline analysis, không còn là hướng chính.
- Metric: IoU/Dice mask, PSNR/MSE background, CLIP-Whole/Edited, runtime; có failure cases cho add object/style/global edit.

Bước tiếp theo:

- Chỉ quay lại hướng này nếu còn thời gian và cần phân tích chất lượng mask.
- Không đưa SAM 3 vào pipeline realtime chính của đề tài.

2026-06-05 — [3c] Chốt bộ độ đo đánh giá SwiftEdit/PieBench

Môi trường: Mac M4 (MPS); .venv

Công việc đã làm:

- Ghi rõ bộ metric chính: PSNR/MSE trên vùng không edit ($1 - \text{mask}$), CLIP-Whole và CLIP-Edited theo `edit_prompt`, runtime theo thời gian gọi `edit_image()`.
- Cập nhật QA.md, SwiftEdit_Overview.md, SwiftEdit_DeTai_CS2309.md, README.md để thống nhất nguồn công bố/công nhận: PIE-Bench/PnP Inversion ICLR 2024; SwiftEdit CVPR 2025 Table 1; CLIP/CLIPScore.
- Sửa `scripts/piebench_metrics.py`: `clip_whole` dùng `edit_prompt` thay vì `src_prompt`, khớp metric `clip_similarity_target_image` của PIE-Bench.

Kết quả:

- Định nghĩa metric trong tài liệu khớp pipeline hiện tại: background preservation = PSNR/MSE vùng nền; edit fidelity = CLIP toàn ảnh/vùng edit với target prompt.
- source `.venv/bin/activate && python -m compileall -q scripts/piebench_metrics.py` chạy OK.

Bước tiếp theo:

- Chạy lại smoke eval hoặc subset PIE-Bench sau khi có dữ liệu đầy đủ để cập nhật lại `metrics.csv` theo định nghĩa CLIP-Whole mới.
-

2026-06-05 — [3c] Xây pipeline đánh giá PieBench (metrics + eval)

Môi trường: Mac M4 (MPS); .venv; torchmetrics 1.9.0

Công việc đã làm:

- Viết scripts/piebench_utils.py (đọc mapping_file.json, RLE mask decode, chọn subset), piebench_metrics.py (PSNR/MSE vùng background, CLIP-Whole/Edited qua torchmetrics CLIP ViT-L/14), run_piebench_eval.py (loop edit_image + ghi metrics.csv, có resume). Thêm download_piebench.sh + create_piebench_smoke.py (2 ảnh demo test pipeline khi chưa tải form). Thêm cell 3b/3d vào notebook phase3. Ghi QA.md mục 5 về bộ độ đo.

Kết quả:

- Smoke eval 1 mẫu chạy OK trên Mac MPS (~50s/ảnh, CLIP-Whole=18.76, CLIP-Edited=22.64); metrics.csv sinh đúng. PieBench đầy đủ tải qua form PnP Inversion (không có trên git). Độ đo theo chuẩn ICLR 2024 (PIE-Bench) + SwiftEdit CVPR 2025 Table 1.

Task README đã đánh [x]:

- (không đánh task README)

Vấn đề / cách xử lý: Thiếu torchmetrics trong venv cũ → cài + script báo lỗi sớm trước khi load model. Bug permute tensor→PIL và mask broadcast 512×512 → fix bằng resize + Image.open.

Bước tiếp theo:

- Tải PieBench đầy đủ qua form; chạy 50-100 mẫu trên Colab T4; so sánh Table 1 paper

2026-06-05 — [2a. Mac] Runtime cùng preset dog — Mac MPS 30.1s vs T4 1.3s

Môi trường: Mac M4 (MPS); pyenv 3.12.10; .venv

Công việc đã làm:

- Chạy preset 02.jpg dog→dog with mouth opened trên Mac MPS (cùng prompt Colab T4)

Kết quả:

- Edit 'dog' -> 'dog with mouth opened' in 30.1s (inference-only, model đã load)
- Lưu: results/notebook/nb_dog_dog_to_dog_with_mouth_opened.png
- Cùng ảnh/prompt với Colab T4 (2026-06-04)

Bảng runtime (preset 02 . jpg, 512×512, inference-only):

	Paper A100	Colab T4	Mac M4 MPS
edit_image	~0.23 s	1.3 s	30.1 s
Tốc độ tương đối	1× (ref.)	~5.7× chậm hơn A100	~131× chậm hơn A100; ~23× chậm hơn T4

Ghi chú báo cáo: Mac MPS dùng được cho demo/ablation nhỏ; benchmark lớn (PieBench) → Colab T4.

Task README đã đánh [x]:

- Bảng runtime 3 cột: **Mac MPS | Colab T4 | Paper A100**

Bước tiếp theo:

- PieBench 50 mẫu Colab; ablation hyperparameter Mac

2026-06-04 — [2b. Colab] Inference Colab T4 — dog preset 1.3s

Môi trường: Google Colab — Tesla T4 (Colab extension)

Công việc đã làm:

- Chạy notebook CS2309_SwiftEdit_test trên Colab T4 (extension): preset dog→dog with mouth opened (assets/imgs_demo/02.jpg)

Kết quả:

- Edit 'dog' -> 'dog with mouth opened' in 1.3s (chỉ edit_image, model đã load)
- Lưu: /content/CS2309.CH201/results/notebook/nb_dog_dog_to_dog_with_mouth_opened.png
- Ảnh: assets/imgs_demo/02.jpg — preset dog / dog with mouth opened

So sánh runtime (cùng SwiftEdit 512×512, khác preset/ảnh):

Môi trường	Thời gian	Ghi chú
Paper A100	~0.23 s	Báo cáo gốc
Colab T4	1.3 s	Phiên này — inference-only
Mac M4 MPS	~91 s	Demo woman→Taylor Swift (có thể gồm overhead MPS lần đầu)

Task README đã đánh [x]:

- Ghi GPU name (T4/A100) và runtime Colab
- Chạy demo CUDA — kết quả khớp Mac
- Cài requirements.txt (CUDA)

Vấn đề / cách xử lý: Không dùng Drive — weights/HF trên /content

Bước tiếp theo:

- Ghi bảng runtime 3 cột Mac|Colab T4|Paper; PieBench batch Colab

2026-06-04 — [2b. Colab] Notebook Colab extension: GPU, upload, stack HF mới

Môi trường: Colab extension (Cursor/VS Code) + Colab web (T4)

Công việc đã làm:

- notebooks/CS2309_SwiftEdit_test.ipynb
 - Metadata accelerator: GPU, colab.gpuType: T4 (Colab web tự chọn T4)
 - Cell 1: _check_colab_gpu() (nvidia-smi) trước clone/tải weights; hướng dẫn extension New Colab Server → GPU → T4
 - Cell Setup: pip install -U -r requirements.txt — không pin numpy==1.26.4 trên Colab; kiểm tra transformers≥4.46, diffusers≥0.32, import EncoderDecoderCache
 - Upload ảnh: ô Đường dẫn + Upload to Colab (Explorer); nút demo 02.jpg; ấn files.upload() / FileUpload trên extension (widget rỗng value={})
- SwiftEdit/requirements.txt: stack mới transformers≥4.46, diffusers≥0.32, peft, accelerate — bỏ pin torch cu118 / 2.2.1
- requirements-mac.txt: đồng bộ stack mới; numpy≥1.26

- **SwiftEdit/models.py**: `torch.load(..., weights_only=True)` cho `ip_adapter.bin`

Kết quả:

- Patch để Colab extension **có thể chạy** sau quy trình: Restart kernel → cell 1 → Setup → Load models → Áp dụng ảnh (path) → inference
- Chưa ghi nhận inference Colab T4 thành công end-to-end trong phiên này — cần chạy lại sau khi push/pull notebook mới

Task README đã đánh [x]:

- (chưa đánh [x] — chờ xác nhận chạy inference Colab OK)

Vấn đề / cách xử lý:

Lỗi	Nguyên nhân	Cách xử lý
Colab chưa có GPU	Server CPU / Auto Connect	Extension: New Colab Server → T4 ; web: Runtime → T4
numpy.dtype size changed	Hạ numpy==1.26.4 trên Colab có torchvision build NumPy 2	Bỏ pin numpy trên Colab
EncoderDecoderCache ImportError	Colab diffusers mới + transformers==4.37 cũ	Nâng stack (transformers≥4.46, diffusers≥0.32), không hạ về 0.22
Upload widget rỗng / files.upload disabled	Colab extension không có browser session Colab	Upload to Colab → dán /content/... hoặc nút Dùng ảnh demo

Bước tiếp theo:

- Restart kernel → Setup → Load → chạy preset hoặc upload path → ghi runtime inference T4 (s) vào nhật ký
- Đánh [x] README mục Colab khi có bằng chứng (ảnh output + torch/GPU name)

2026-06-04 — [2a. Mac] Notebook test, tài liệu Colab/T4, ghi chú runtime

Môi trường: Mac M4 (MPS); Jupyter .venv

Công việc đã làm:

- Notebook notebooks/CS2309_SwiftEdit_test.ipynb (preset, upload ipywidgets 8, inference)
- README: hướng dẫn cài đặt/pyenv/scripts; .gitignore loại weights
- Ghi nhận: Mac MPS ~91s vs paper A100 ~0.23s; Colab T4 VRAM 15GB đủ inference; Drive ~15GB chật (weights 9.6GB+HF)

Kết quả:

- Notebook chạy OK trên Mac (.venv); upload widget sửa tuple ipywidgets 8
- Chưa chạy Colab thực tế — đã ghi kế hoạch T4 + lưu weights Drive trong đề tài/README

Task README đã đánh [x]:

- Tạo notebook CS2309_SwiftEdit_test.ipynb

Vấn đề / cách xử lý: matplotlib thiếu kernel → %pip trong notebook; FileUpload value tuple (ipywidgets 8)

Bước tiếp theo:

- Chạy thử Colab T4: IN_COLAB=True, weights trên Drive; đo runtime inference-only; PieBench batch
-

2026-06-04 — [2a. Mac] Setup Mac (pyenv/venv) và chạy demo SwiftEdit

Môi trường: Mac M4 (MPS); pyenv 3.12.10; .venv

Công việc đã làm:

- Clone SwiftEdit; pyenv 3.12.10 + .venv + requirements-mac.txt (PyTorch MPS)
- Tải swiftedit_weights (~9.6GB); patch infer.py (get_device mps); patch models.py (SD2.1 mirror Manojb, map_location CPU)
- Tải HF: sd-turbo, Manojb/stable-diffusion-2-1-base, h94/IP-Adapter image_encoder
- Bổ sung tài liệu: §1.3 pipeline cũ, ảnh assets/pipeline/, QA.md

Kết quả:

- Demo woman→Taylor Swift OK; output SwiftEdit/result_woman->Taylor Swift.png; ~91s/ảnh trên MPS
- Scripts: scripts/run_swiftedit.sh, download_swiftedit_weights.sh

Task README đã đánh [x]:

- Cài PyTorch Mac (MPS) — **không** dùng bản CUDA
- Cài dependencies còn lại

- Clone repo SwiftEdit
- Tải checkpoint → `swiftdit_weights/`
- Sửa `device = "mps"` trong code
- Chạy demo `assets/imgs_demo` thành công
- Ghi thời gian/ảnh và RAM peak (Activity Monitor)

Vấn đề / cách xử lý: stabilityai/stable-diffusion-2-1-base 401 → mirror Manojb IP-Adapter image_encoder timeout → tải riêng model.safetensors ip_adapter.bin load CUDA → `map_location=cpu`

Bước tiếp theo:

- Ghi RAM peak (Activity Monitor); setup Colab notebook
- Ablation hyperparameter; PieBench eval trên Colab

2026-06-01 — [0. Khởi tạo project] Khởi tạo cấu trúc đề tài

Môi trường: Mac M4

Công việc đã làm:

- Tạo README, đề tài chi tiết, skill Cursor hỗ trợ nhật ký

Kết quả:

- Repo CS2309.CH201 sẵn sàng; skill sync README + NHAT_KY + §8.1

Task README đã đánh [x]:

- *(không đánh task README)*

Bước tiếp theo: Clone SwiftEdit; cài env Mac MPS
